Chess

→ Notation styles:

→ Chosen notation for input file

① Standard Algebraic ——→ Be5, Nf3
- ↳ Only end destinations
- ↳ Code needs to figure out which piece to choose
  - ↳ Requires thinking (Reverse engineering — see which B or N can move there)
  - ↳ Lots of opportunities for optimization

② Figurine Algebraic Notation
- ↳ Not gonna work for this
  - ↳ Cool if I had a GUI

③ Long Algebraic Notation ——→ e2e4 Nb1-c3
- ↳ Start & end coordinates
- ↳ Quite easy for the model
- ↳ Finds piece
  - ↳ Validates move
    - ↳ Makes move
- ↳ Would be smart for the LLM to change to this

④ Abbreviated Algebraic Notation ——→ exd, ed
- ↳ Seems maybe a bit too much for scope
- ↳ LLM will need some training to understand this

⑤ Descriptive Language → Natural language, Pawn to e3
- ↳ Way out of scope → this is ML

⑥ FEN ⟶ rnbqkbnr/pppppppp/...

↳ Entire board each time

↳ Not for my input file

↳ LLM might use it

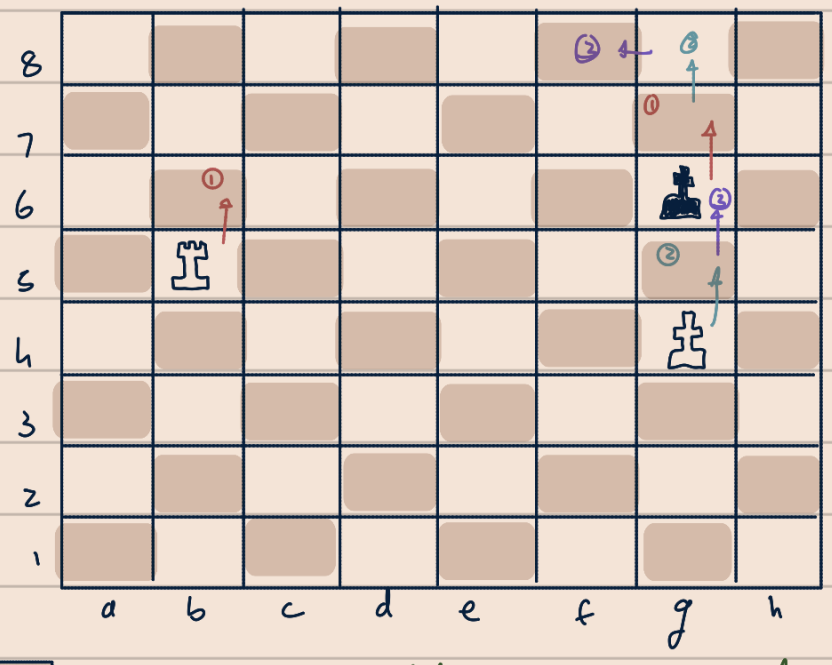
⟶ Data sets:

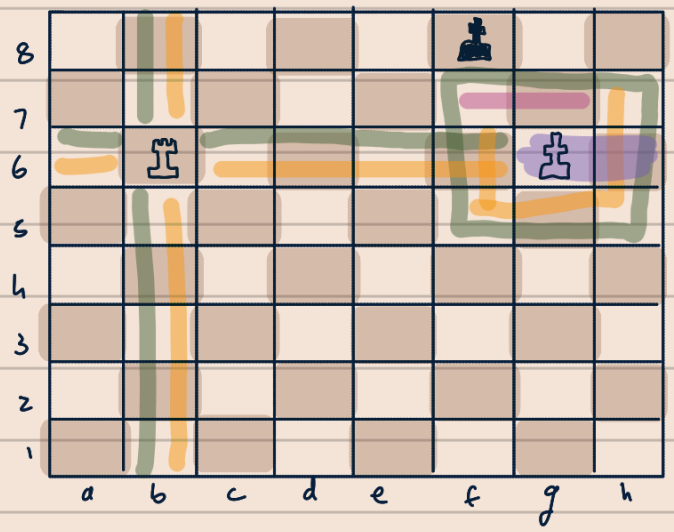
① Single Rook endgame:

Rb6 kg7

kg5 kg8

kg7 kf8





All Possible move for white 12+8=20

All legal moves for white 12+6=18

Illegal moves 0+2=2

Impossible moves

↳ Rg6, Rh6, Rb6, kg6 ✓✓

18 + 2 = 20

Code must output a textfile:

WRb6; Wkg6; Bkf8 (Current Position of all pieces)

20: Ra6; Rb8; Rb7; Rb5...

18: Ra6; Rb8; Rb7; Rb5...

2: kf7; kg7

Write a script to read textfile & determine this output

Prompt 2: Directional feedback

All moves, 2 are wrong

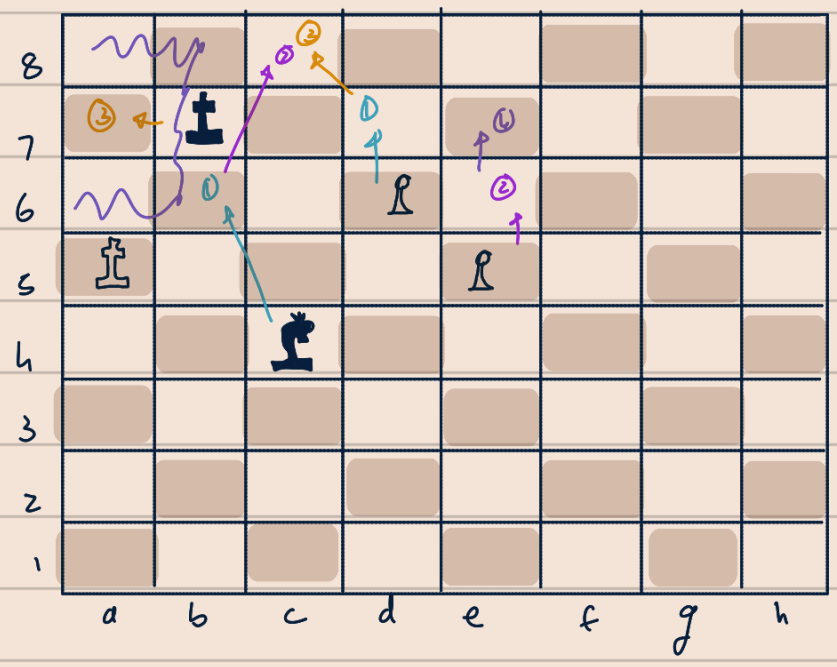
Possible moves, 3 are wrong

Illegal moves, all are correct

Impossible moves given: 2

Dataset 2:

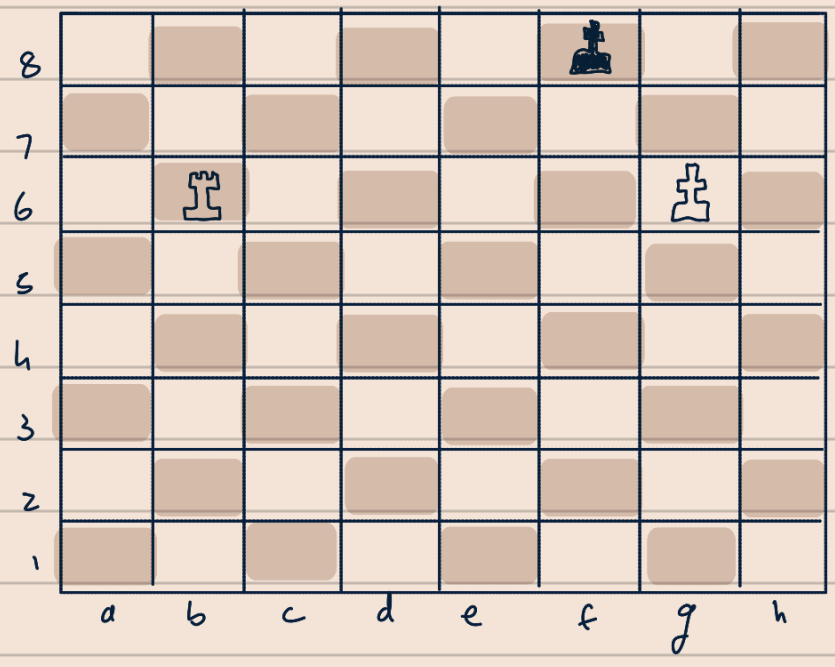
Begin: Wka5; Bkb7; BNc4;
Wc5; Wd6



1. d7 Nb6
2. c6 N c8
3. dxc8 =Q ka7
3. c7 Stalemate

Wka5; Bka7; WQc8; c7

All moves: a8; b8; b7; b6; a6 5
Legal: 0 0
Illegal: 6 a8 b8 b7 b6 a6 5
Impass: a7 1



Experimental Procedure

→ Zed
  ↳ Claude Sonnet 4.6
  ↳ Thinking effort high
  ↳ Profile = Write
→ Python 13.3.9

Response time too long & ran out of prompts?
↳ I have a free student account

→ Begin by Run_D1_C2.3
  ↳ Copy no 1
  ↳ Copy no 2

→ Gemini 3.1
  ↳ Pro

1.2.3 → does not run!!!

Experimental change: Test on Fast branch

→ Prompt 1 : Zero context
→ Prompt 2 : Directional in the loop feedback
→ Prompt 3 : Balance efficiency & logic

Gemini 3 Flash

Repeat prompt 2 till 80% accuracy then prompt 3

Run

D1_C1.1

Time thinking (s)

126

Accuracy

46.66%

Fast 144 → Referenced a Youtube Video & add an extra line in output

Github

- experiments
  - data set 1
    - run 1
      - prompt 1
        - chess.py
        - input.txt
        - output.txt
    - run 2
  - data set 2
- correct outputs → data1.txt, data2.txt, data3.txt
- data sets (code-og) → data1.py, data2.py, data3.py
- helper scripts → prompt2.cpp
- benchmarking
  - lizard
    - for each run
  - runtime
    - windows or cpp? or bash?
    - each run
  - error
    - each run
- docs
  - these notes lol
- Readme
  - contain links to chats

Automate through the 10 runs & every data set
Output a summary textfile for each data set

Also accuracy = $\left(\frac{\sum \text{Correct moves}}{\sum \text{Expected moves}}\right) * 100$

Metrics:

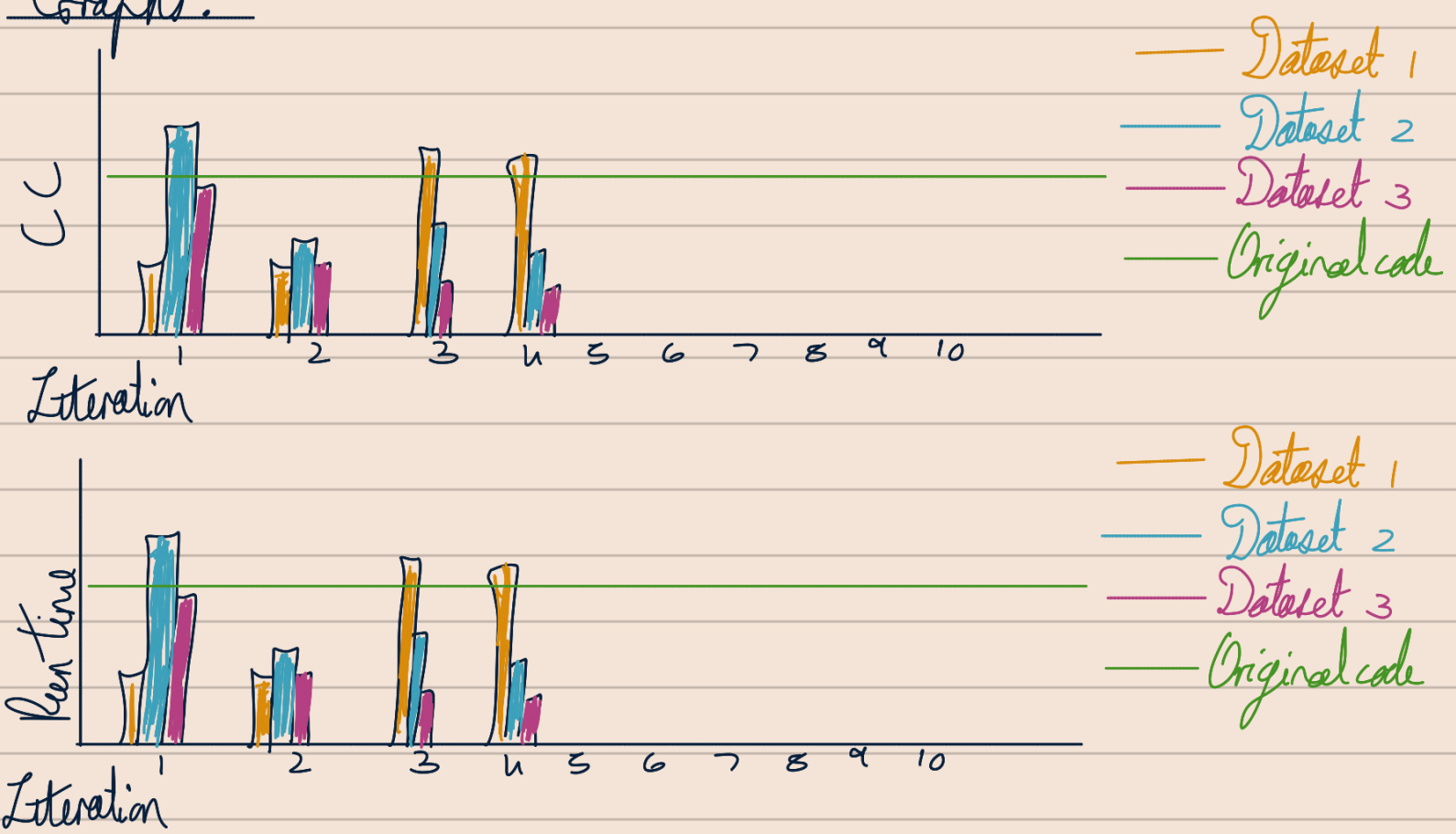
neg marking ↓

Dataset	missed Rule	Accuracy	Lizard	Reg. Rate during opt
1	Background & Research cap.	Implementation of logic	Cyclomatic complexity	Efficiency stress test → Does it compromise logic for efficiency? Or can it Balance
2				
3				

Logical Reliability Structural Quality

Dataset	Build/Compile error	Runtime error
1	/90	/90
2	/90	/90
3	/90	/90

Graphs:



Important

* Internal validity : Directional in-the-loop feedback must be standarised

* Context leakage control: How is it enforced? => Fresh chats

* External Validity: Be real, I have one LLM, 30 end products ✓ claims cannot really be generalised